

F.E.A.R. Project - User Manual

Fully Encrypted Anonymous Routing

Version: 2.0 (v0.6.0) **Author:** Shchuchkin E. Yu.



Table of Contents

1. [Introduction](#)
2. [How F.E.A.R. Protects You](#)
3. [Installation](#)
4. [Getting Started](#)
5. [Rooms, Contacts and Personal Chats](#)
6. [Identity and Verification](#)
7. [Calls](#)
8. [File Transfer](#)
9. [Settings](#)
10. [Console Programs](#)
11. [Running Your Own Relay](#)
12. [Troubleshooting](#)
13. [FAQ](#)

Introduction

F.E.A.R. (Fully Encrypted Anonymous Routing) is an open-source messenger for Windows, Linux and Android. Messages, files and calls are encrypted on your device and decrypted only on the devices of the people you talk to. The server in the middle – the *relay* – passes encrypted data along and cannot read it.

Key Features

- **End-to-end encryption** of messages, files, voice and video (AES-256-GCM)
- **Room keys that follow the room:** the key changes every time someone joins or leaves, so a newcomer cannot read what was said before, and someone who left cannot read what is said after
- **A relay that knows little:** it sees a hash instead of the room name and a fresh random tag instead of your name
- **Group voice and video calls** through the relay, with per-participant keys
- **Personal chats and contacts**, with an encrypted contact list kept on the relay
- **Offline delivery:** a message to a contact who is not online waits on the relay, sealed, until they collect it
- **Identity you can verify:** an Ed25519 key per user, fingerprints that read the same on every platform, a warning when someone's key changes
- **Encrypted backup** of your identity (file or QR code)
- **Optional TLS** to the relay, and optional direct calls (STUN)
- **Your own relay** in one command, or a Docker image
- **Free software:** GPL-3.0 clients, AGPL-3.0 relay

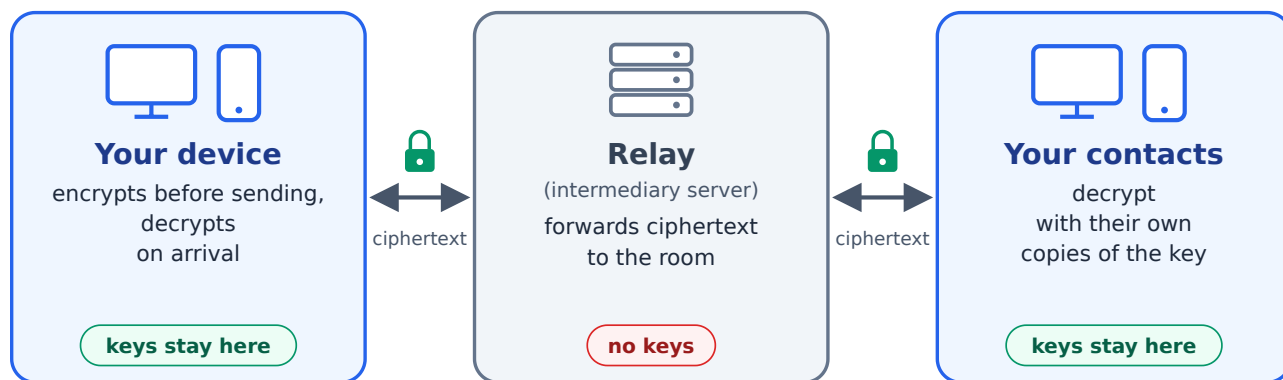
What's New in v0.6.0

- **Room key rotation** on every change of membership (forward secrecy within a room)
- **Metadata privacy:** room names and display names no longer appear on the relay
- **Group calls:** several people at once, each sender with its own key; names under the video tiles
- **Offline inbox** on the relay for messages to contacts who are not online
- **Android notifications without Google services**
- **Noise suppression** and microphone sensitivity; manual video settings
- **Optional TLS** to the relay; optional direct calls via STUN
- **The desktop identity key is encrypted at rest** (system keyring / DPAPI)
- **One desktop window** (the classic interface is gone; everything it had moved over)
- **Complete release archives:** calls, updater and key exchange included

v0.6.0 does not talk to v0.5.x. The wire format changed (hashed rooms, session tags, a new key schedule and call framing). Update every device and the relay together.

How F.E.A.R. Protects You

A short illustrated overview for non-specialists – what is encrypted, where data is kept, what the relay can and cannot see – is in [doc/how-it-works.pdf](#) (English) and [doc/how-it-works.ru.pdf](#) (Russian).



End-to-end encryption: only the participants' devices can open the data

What the relay sees and what it does not

The relay sees	The relay does not see
IP addresses of the connections	Messages, files, voice, video
When and how much each connection sends	Any key
r: + a hash of the room name	The room name itself
A random 16-byte tag per connection, new each time	Display names (they travel inside the encryption)
Your public key and handle, if you register a handle	Who your contacts are (the contact list is encrypted)

With **TLS** enabled, people watching the network (your provider, the owner of a Wi-Fi network) see only an encrypted stream instead of F.E.A.R. frames. TLS hides nothing from the relay itself.

Cryptography

Primitive	Used for
AES-256-GCM	Chat messages, files, voice and video
X25519	Delivering the room key to someone joining (ECDH); the key of a personal chat
Ed25519	Your identity: signed identity announcements, signed key delivery
BLAKE2b	Key derivation, the room hash on the wire, fingerprints

Room keys and rotation

A room has a key (K_{room}). When you **create** a room, your client draws a fresh one. When you **join**, a member delivers it to you over X25519, signed with their identity key so that nobody in between can substitute their own.

Every time the membership changes, one member – chosen the same way by everyone – draws a new generation of the key and sends it to each member sealed with that member's own key. Messages are encrypted under per-epoch keys derived from the current generation. As a result:

- someone who joins cannot read what was said before they arrived;
- someone who leaves cannot read what is said after they left.

Calls

Calls run in a separate program and over a separate connection to the same relay. Each participant encrypts what they send under their own key, derived for that call; announcements are authenticated, and replayed packets are dropped.

Personal chats

A personal chat uses a key derived from your identity and your contact's identity (X25519), so it works even if the other person is not online when you open it. Its identifier on the relay is derived under that key: the relay cannot tell from it that the chat is personal, nor work out whose it is.

What F.E.A.R. does not do

- **No post-compromise security.** There is no Signal-style ratchet: someone who steals a room key reads that room until the next rotation.
- **IP addresses are visible to the relay.** Use VPN or Tor if that matters to you, or run your own relay.
- **The room hash is not secret.** A determined relay operator can guess common room names ("general") by hashing them.
- **Identity announcements** (display names and the tags they belong to) are encrypted under the room's founding key, which every member who ever joined holds. A former member who also had the relay's traffic could learn who is in the room – but not what is said.
- **Your device is trusted.** Malware running as you can read what you can read.

Installation

Desktop (Windows and Linux)

Download the archive for your system from [Releases](#):

- `fear-windows-x86_64-v0.6.0.zip`
- `fear-linux-x86_64-v0.6.0.zip`

Extract it into a folder of its own and keep the layout as it is:

```
fear_gui(.exe)      – the application
bin/                – programs the application starts:
  fear              – console client and relay
  audio_call        – voice calls
  video_call        – video calls
  updater           – updates
  key-exchange      – manual key exchange
  updater.conf, cacert.pem
doc/manual.pdf      – this manual
LICENSE, LICENSE.GPL-3.0, LICENSING.md, README.txt
```

Windows: run `fear_gui.exe`. The build is not code-signed; SmartScreen will warn the first time – click "More info", then "Run anyway".

Linux: run `./fear_gui`. The archive is built on Ubuntu 22.04 and runs on Ubuntu 22.04+ and Debian 12+. The libraries it needs are listed in `README.txt` with the exact `apt install` line. Video (FFmpeg) and window (SDL3) libraries are built into `video_call`.

Android

Install the APK from [fear-mobile Releases](#). Allow installation from unknown sources when Android asks.

Updating

Desktop: **Check for updates** in the menu runs the updater. It downloads the new archive, checks its **Ed25519 signature** and refuses anything unsigned or signed by another key, then unpacks it over the installation. Android checks for updates from its menu as well.

Building from source

See [BUILD.md](#). In short: `./build.sh deps && ./build.sh` on Linux, `build.bat` on Windows (with the libraries in `lib/`).

Getting Started

First run

On first start the application creates your **identity** – an Ed25519 key pair – and asks for a display name. Make an **encrypted backup** of the identity soon (menu → **Export identity...**): without it, a lost device means a lost identity.

Connecting to a room

Menu → **Connect to room...** opens the connection dialog:

Field	Meaning
Server	A public relay (Netherlands, Russia) or your own: choose "Custom server..." and type the address
Port	8888 unless the relay says otherwise
Room	Any name; everyone who types the same name on the same relay meets in the same room
Name	Your display name in this room
Mode	Auto (default): create the room if it is empty, otherwise join it. Create , Join and Manual key for special cases

The dialog also shows whether your identity has a **handle** on this relay (`nickname@server`). Register one so that others can add you as a contact.

The window

- **Sidebar** – your contacts and your groups. **+** adds a contact, joins a room or creates a new one.
- **Chat** – messages with a date above each day, the participants (click the room title), call buttons, the attach button.
- **Search** – full-text search through local history.
- **Tray icon** – closing the window hides it and keeps you connected. Use **Quit** to exit.
- **Theme** – light or dark, from the menu; the choice is remembered.

History is stored locally (SQLite in your user data folder). The relay keeps no chat history.

Rooms, Contacts and Personal Chats

Groups (rooms)

Anyone who knows the relay and the room name can enter the room and receive its key from a member. A room is private in the sense that nobody outside it can read it – not in the sense that the name is a password. Use a name that is hard to guess for a group you want to keep to yourselves, or **Manual key** to require a key shared out of band.

Contacts

Add a contact by handle (`nickname@server`). The contact list is stored on the relay encrypted with a key derived from your identity, so it follows you to another device after you restore your identity there. The relay cannot read it.

Personal chats

Click a contact to open a personal chat. Its key is derived from both identities; the other person does not need to be online.

Offline inbox

If your contact is not online, your message waits on the relay in a sealed **inbox** addressed by a blinded value only the two of you can compute. Clients collect waiting messages automatically (on the desktop every 20 seconds, across all contacts at once). How long messages wait is the relay operator's choice: 30 days by default, or none at all. When a message could not be delivered within that time, the client says so rather than pretending it arrived.

Identity and Verification

Fingerprints

Every identity has a fingerprint: `40:f6:8f:4a:d2:4e:57:5b` (BLAKE2b, 8 bytes). The short form `name#40f68f4a` is its first 4 bytes. The value is the same on Windows, Linux and Android – **compare it with your contact over a channel you trust** (in person, a call) to be sure nobody is in between.

Trust On First Use

The first key seen for a contact is remembered. If it later changes, the application shows a warning. It can be a reinstall without a backup – or an attack. Ask the person before you trust the new key. Menu → **Trusted keys** lists remembered keys and lets you mark them as verified.

Backup and restore

Menu → **Export identity...** writes an encrypted `.fbk` file or shows a QR code; the password must be at least 12 characters. **Import identity...** restores it on another desktop or on Android (and the other way round). After a restore, your handle is found again automatically.

Storage

Platform	Identity	Protection
Linux	<code>~/.fear/identity</code>	Secret key wrapped by a key in the system keyring (libsecret)
Windows	<code>%APPDATA%\fear\identity</code>	Secret key protected with DPAPI
Android	app storage	EncryptedFile backed by the Android Keystore

Without a running keyring (headless Linux) the desktop falls back to a file readable only by you, and says so.

Calls

Starting and answering

Press the **phone** (voice) or **camera** (video) button in a room. Everyone in the room receives an invitation; answering joins the call. Calls go through the same relay as the chat.

- **Voice:** up to 8 voices are mixed at once; the others are heard as soon as they speak.
- **Video:** the desktop shows up to 4 pictures at once – the active speaker large, the others in a strip. Names appear under the tiles (the desktop video window transliterates Cyrillic names, its font is ASCII-only).
- **Hang up** from the call window or by closing it; the call program wipes its keys on exit.

Quality

Preset	Resolution	FPS	Bitrate
Low	320×240	15	200 kbit/s
Medium	640×480	25	500 kbit/s
High	1280×720	30	1500 kbit/s

The desktop steps down when packets are lost or the delay grows; the Android app lowers its bitrate when the network does not keep up and returns to your setting when it clears. Width, height, frame rate and bitrate can be set by hand in **Settings → Video**.

Direct calls (optional)

With a **STUN server** set in **Settings → Audio**, calls try a direct path first. It is shorter and the relay operator does not see the call stream – but **your IP address is revealed to the people you call and to the STUN server**. The field is empty by default, which means always through the relay.

File Transfer

Use the **attach** button in a chat. The recipient sees the file name and size and chooses **Accept** (saved to `Downloads`), **Save as...** or **Reject**; **Settings → Privacy** can accept files automatically. Files are encrypted like messages and checked for integrity on arrival.

Settings

Tab	What is there
General	Path to the console client; TLS to the relay and an optional certificate pin

Tab	What is there
Chat	Chat font family and size
Audio	Default devices, noise suppression (off/low/medium/high), microphone sensitivity, STUN server
Video	Default camera, resolution, frame rate, bitrate
Privacy	Whether notifications show the message text, automatic acceptance of files
Identity	Whether you have an identity key, its fingerprint, how many keys you trust

Noise suppression on the desktop cuts low hum and closes a gate between words: background noise you hear in pauses goes away; it is not removed from under the voice. On Android the system noise suppressor is used, and can be switched off on phones where it cuts quiet speech.

Console Programs

The desktop application starts these itself. They are also useful on their own – for servers, scripts and testing.

fear – console client and relay

```
fear --version
fear genkey                # print a random room key
fear gen-identity          # create ~/.fear/identity
fear server [--port N] [--inbox off|30d|Nh] [--tls-cert FILE --tls-key FILE]
fear client --host HOST --port N --room ROOM [--name NAME]
                  [--auto | --create | --join | --key-file FILE]
                  [--identity-file FILE] [--no-sign] [--tls] [--tls-pin SHA256HEX]
```

Client option	Meaning
--auto	Ask the relay: empty room – create it, otherwise join (what the GUI does by default)
--create	Create the room with a fresh key
--join	Receive the key from a member (X25519, signed)
--key-file FILE	Read a pre-shared key from a file; without any of these, the key is read from stdin
--tls, --tls-pin	Connect over TLS; with a pin, accept only that certificate

A key passed in by hand only founds the room: it still rotates whenever someone joins or leaves.

In the chat: type and press Enter to send; `/sendfile PATH` sends a file; Ctrl+C exits.

audio_call and video_call

Normally started by the application for a call in the room. On their own:

```
audio_call listdevices
video_call listdevices
video_call relay HOST PORT --room ROOM --name TAG --call-id HEX32 [options]
```

Useful options: `--quality low|medium|high`, `--width`, `--height`, `--fps`, `--bitrate`, `--camera DEVICE`, `--no-camera` (receive only), `--no-video`, `--no-audio`, `--mic-gain dB`, `--noise-suppress off|low|medium|high`, `--stun HOST[:PORT]` (port 3478 by default). The key is read from stdin or `--key-file`.

updater

Checks the latest release, verifies its signature and installs it over the current folder. Run by **Check for updates** in the application.

key-exchange

An interactive X25519 tool for passing a room key to someone over any channel when you cannot use **Join**: both sides generate key pairs, exchange public keys, one encrypts the room key for the other.

Running Your Own Relay

Your own relay keeps the metadata – who connects when, and from where – in your hands.

From the archive

```
./bin/fear server --port 8888
```

The relay needs no keys; it cannot decrypt anything. It keeps its state (handles, encrypted contact lists, the offline inbox) in `fear-server.sqlite` in its working directory.

Docker

```
docker run -d --restart=unless-stopped --name fear-server \
  -p 8888:8888 -v fear-data:/var/lib/fear \
  ghcr.io/shchuchkin-pkims/fear-server:latest
```

Options

Option	Meaning
<code>--port N</code>	TCP port, 8888 by default
<code>--inbox 30d Nh off</code>	How long the offline inbox keeps messages. <code>off</code> stores nothing and discards what was stored. Per recipient at most 200 messages or 5 MB
<code>--tls-cert FILE --tls-key FILE</code>	Accept TLS connections. Clients then need <code>--tls</code> (or the TLS box in Settings)

The relay accepts up to 16 connections per IP address and 100 in total, and closes a connection that has been silent for 4 minutes (clients send a keep-alive at least once a minute).

On the desktop, **Run a relay here...** starts a relay on your machine for a LAN.

Administration

`fear_admin` (Linux, built from source with Qt 6) works on the relay's database on the same machine: registered handles, stored blobs, the inbox, blocked keys, live sessions. It has no network interface by design.

Troubleshooting

Cannot connect

- Check the server address and port; check that the relay is running and the port is open.
- If the relay uses TLS, enable TLS in **Settings → General**; if it does not, disable it.

Connected, but nobody appears, names stay as short tags, messages do not arrive

- Everyone must run v0.6.0 – including the relay.
- The console client says it outright: *"Cannot read who else is here... You are in this room with a different room key"*. Everyone should leave and rejoin, or pick a room name nobody is using yet.

A contact's key changed

- Ask them over another channel whether they reinstalled. Compare fingerprints before trusting the new key.

No sound or picture in a call

- Check the devices in **Settings → Audio / Video** and the system permissions for the microphone and camera.
- On Linux, `bin/video_call listdevices` shows what the program sees.

Video stutters

- Wi-Fi at 2.4 GHz is the usual cause; a cable or 5 GHz helps. Lower the quality in **Settings** → **Video**.

The settings show no devices, or the application cannot find its programs

- Keep the archive layout: the programs must be in `bin/` next to the application.

Linux: the application does not start

- Install the packages listed in `README.txt` . Run `./fear_gui` from a terminal to see which library is missing.

FAQ

Q: Can the relay read my messages?

A: No. It has no keys. It routes encrypted frames by the room hash.

Q: Does anything keep my messages?

A: Your own devices keep local history. The relay keeps nothing of a live chat; the **offline inbox** keeps sealed messages for contacts who are not online, for as long as the operator allows (30 days by default), and deletes them once collected.

Q: Is F.E.A.R. anonymous?

A: It hides content, room names and display names from the relay. It does not hide that you connect, from which IP address, or when and how much you send. For that, use VPN or Tor, or your own relay.

Q: Do I need to change room keys?

A: No. The room key changes by itself whenever someone joins or leaves.

Q: How many people can be in a room?

A: The relay accepts up to 100 connections. In a call, up to 8 voices are mixed and the desktop shows up to 4 pictures at once.

Q: Does it work with v0.5?

A: No. v0.6.0 changed the wire format. Update all devices and the relay together.

Q: Is there a mobile app?

A: Yes, for Android: [fear-mobile](#). It speaks the same protocol, including rotation, group calls and the offline inbox, and shows notifications without Google services.

License

F.E.A.R. is free software. The relay and console client (`client-console/` , `identity/` , `Dockerfile` , `web/server.js`) are licensed under the **GNU AGPL-3.0-or-later**; the clients (`gui/` , `audio_call/` , `video_call/` , `key-exchange/` , `updater/` , `web/public/` , the Android app) under the **GNU GPL-3.0-or-later**. See [LICENSING.md](#) for the full mapping, and [LICENSE](#) and [LICENSE.GPL-3.0](#) for the texts.

Contact

GitHub: <https://github.com/shchuchkin-pkims/fear> **Issues:** <https://github.com/shchuchkin-pkims/fear/issues> **Project site:** <https://fear-project.ru>

Reporting Bugs

1. Open a GitHub issue
 2. Include: F.E.A.R. version, OS, steps to reproduce, expected vs actual behaviour
 3. Attach logs if available (run the program from a terminal)
-

Stay Anonymous. Stay Secure. Shchuchkin E. Yu. F.E.A.R. Project